

LFX Mentorship Summer 2026

Extending Language & Library Support in CBOMkit

A Comprehensive Technical Mentorship Application

Project: cbomkit/sonar-cryptography : C/C++ Support & OpenSSL Detection

Host Organization: Post-Quantum Cryptography Alliance () / Linux Foundation

Term: June 9 – August 29, 2026 (12 weeks, full-time, remote)

Date Prepared: May 8, 2026

This document covers: (1) Codebase Architecture & C/C++ Integration Plan | (2) 12-Week Milestone Timeline | (3) Validation Projects | (4) Library Research Table | (5) OPA Policy Tests | (6) Cover Letter & Technical Proposal | (7) Success Metrics

Section 1: Codebase Analysis & C/C++ Integration Plan

1.1 Verified Architecture: sonar-cryptography (CBOMkit-hyperion)

The following architecture is drawn directly from the official [blog post](#) (March 2026) and the [cbomkit/sonar-cryptography](#) repository on GitHub. All five CBOMkit components are described below with their verified roles.

1.1.1 Five-Component CBOMkit Ecosystem (Verified)

Component	Codename (Historical IBM)	Role
sonar-cryptography	Hyperion	SonarQube plugin : detection engine. AST-based crypto asset identification for Java, Python, Go. Produces CycloneDX CBOM JSON.
cbomkit-lib	Hyperion (library layer)	Standalone Java library wrapping sonar-cryptography. Enables scanning without a running SonarQube server via SensorContextTester.
cbomkit-action	:	GitHub Action (Docker/UBI8 + OpenJDK 21) packaging cbomkit-lib for CI/CD pipeline integration.
cbomkit	Coeus + Themis + Mnemosyne (historical internal; not used in 2026 public materials)	Full-stack app with REST API, Vue.js UI, PostgreSQL persistence, compliance evaluation, and OPA policy support.
cbomkit-theia	Theia	Go CLI scanning container images and local filesystems for deployed cryptographic artifacts (certs, keys, secrets, OpenSSL config).

1.1.2 Internal Maven Submodule Structure of sonar-cryptography

The plugin is organized as a multi-module Maven build (Java 17). Each submodule has a clearly separated responsibility:

- `engine/` : Language-agnostic detection framework. Defines `DetectionStore`, `DetectionExecutive`, and `IDetectionRule<T>`. This module does NOT parse source code; it operates on ASTs provided by language-specific Sonar plugins.
- `java/`, `python/`, `go/` : Language-specific modules, each providing: detection rules, an AST translator (`ITranslator`), and a `CheckRegistrar` that integrates with the corresponding SonarQube language plugin.
- `mapper/` : Converts raw `DetectionStore` entries into the `INode` semantic hierarchy: Algorithm, Key, Protocol, Mode, Padding, KeyLength, DigestSize, etc.
- `enricher/` : Adds OIDs, default parameters, and algorithm-specific metadata not present in source code (e.g., adds OID for AES-128-GCM, assigns default mode when none is explicit).

- `output/` : Generates CycloneDX v1.6 CBOM JSON. Each crypto asset becomes a component of type `cryptographic-asset` with file/line evidence and dependency relationships.
- `common/` : Shared utilities; observer-pattern infrastructure for event-driven communication between pipeline stages.
- `sonar-cryptography-plugin/` : Plugin entry point. Registers language extensions via `CryptographyPlugin.java` (implements `Plugin` interface). The `OutputFileJob` `PostJob` collects detections from all language Aggregators through `ScannerManager` and writes `cbom.json`.

1.1.3 Detection Pipeline (Five Verified Stages)

1. **Stage 1 : Detection:** Detection rules (built with `DetectionRuleBuilder`) are applied to AST nodes produced by the SonarQube language plugin. When a rule matches, the result is recorded in a `DetectionStore` : a tree structure capturing the location, extracted parameter values, and any child detections. `DetectionExecutive` manages the lifecycle of a detection run per source file and emits a `Finding` event for each completed match.
2. **Stage 2 : Translation:** An `ITranslator` converts `DetectionStore` entries into the language-independent `INode` semantic hierarchy: `Algorithm`, `Key`, `Protocol`, `Mode`, `Padding`, `KeyLength`, `DigestSize`. Each node carries an origin attribute (`detected` / `inferred` / `enriched`). This step also handles the Hook system : `EnumHook`, `MethodInvocationHook`, and `HookRepository` : which captures indirect parameters (e.g., key sizes stored in `RSAPublicKeyGenParameterSpec` rather than passed directly to the primary method call).
3. **Stage 3 : Reorganization:** A `Reorganizer` applies a set of reorganizer rules to restructure the translated `INode` tree : promoting child nodes, merging related detections, and applying precedence rules. This step is explicitly called in the `initiate()` function defined in `LANGUAGE_SUPPORT.md` and is distinct from both translation and enrichment.
4. **Stage 4 : Enrichment:** The `Enricher` traverses the reorganized `INode` tree and adds derived metadata not present in source code: `OIDs` (e.g., `OID` for `AES-128-GCM`), default parameters (e.g., default mode when none is specified), and algorithm-specific properties from external knowledge sources.
5. **Stage 5 : Output:** `CBOMOutputFile` serializes the enriched `INode` tree into a CycloneDX v1.6 CBOM JSON document. Each crypto asset becomes a component of type `cryptographic-asset` with file path, line number evidence, and dependency relationships to related assets.

1.1.4 The Four Generic Classes Required for Any New Language Module

As documented in `LANGUAGE_SUPPORT.md`, every language module must implement four SonarQube generic classes through a language-specific parser:

Generic Class	Purpose	sonar-cxx Status (as of Feb 2026)
R (Rule)	Represents a check/rule instance	Supported
T (Tree)	Represents the AST tree node type	Supported
P (Plugin)	Represents the language plugin context	Supported
S (Symbol)	Symbol table for type resolution and traversal	MISSING : PR #3013 raised to sonar-cxx by chmodshubham; targeted for sonar-cxx 2.3.0

KEY FINDING (GitHub Issue #374, February 22, 2026): Community contributor `chmodshubham` has already implemented C/C++ support in a fork and covered

approximately 90% of OpenSSL 3.x cryptographic assets (both libcrypto and libssl). The blocking dependency is sonar-cxx 2.3.0 which will include Symbol class support via PR #3013. The mentee must build on this prior work, resolve the PR merge, write the test suite, and integrate with cbomkit-lib.

1.2 Exact Steps to Add C/C++ Support

The following is a concrete, step-by-step implementation plan derived from the existing architecture, the prior work in Issue #374, and the documented requirements in LANGUAGE_SUPPORT.md.

Step 1: Verify and Contribute to sonar-cxx Symbol Support

- Track sonar-cxx PR #3013 (raised by chmodshubham to SonarOpenCommunity/sonar-cxx). Coordinate with sonar-cxx maintainers to ensure the Symbol class and required utility methods are merged into the 2.3.0 milestone.
- If the PR is not merged before the mentorship start (June 9), create a Maven local dependency using the fork's SNAPSHOT jar to proceed with integration testing.
- Required methods beyond Symbol: review the LANGUAGE_SUPPORT.md 'Identifying the four classes' section and verify that the sonar-cxx fork correctly implements all required interface contracts (getType(), getSymbol(), etc.).

Step 2: Create the cpp/ Maven Submodule

Mirror the structure of the java/ module:

```
sonar-cryptography/
  cpp/
    src/main/java/com/ibm/plugin/languages/cpp/
      CppCheck.java           // extends AbstractCheck<CxxTree>
      CppCheckRegistrar.java  // implements CheckRegistrar
      CppLanguage.java        // Language metadata (key, name, file suffixes)
      CppSensor.java          // extends AbstractSensor; registers CppCheck
      rules/openssl/
        EvpCipherRules.java
        EvpDigestRules.java
        EvpPkeyRules.java
        EvpMacRules.java
        SslCtxRules.java
        EvpKeyGenRules.java
        LegacyRules.java      // deprecated API detection (AES_*, DES_*, SHA_*)
      src/test/
        java/ // JUnit 5 tests per rule file
        resources/openssl/ // .c / .cpp test fixture files
    pom.xml
```

Step 3: Implement CppCheck.java

CppCheck extends the `AbstractCheck<CxxTree>` class (from `sonar-cxx`), registers the detection rules for OpenSSL, and delegates to the engine-level `DetectionExecutive` for pattern matching. It is the equivalent of `JavaCheck` in the `java/` module.

Step 4: Wire Up CryptographyPlugin.java

In `sonar-cryptography-plugin/src/main/java/.../CryptographyPlugin.java`, add the `CppCheckRegistrar` to the list of extensions and register the `CppLanguage`. This mirrors how `JavaCheckRegistrar`, `PythonCheckRegistrar`, and `GoCheckRegistrar` are currently registered.

Step 5: Implement CppIndexService and CppScannerService in cbomkit-lib

`cbomkit-lib` requires a new `CppIndexService` (extends `IndexingService`) that discovers C/C++ source files (`*.c`, `*.cpp`, `*.cc`, `*.cxx`, `*.h`, `*.hpp`) in a project directory, and a `CppScannerService` (extends `ScannerService`) that runs the detection rules. These follow the exact patterns of `JavaIndexService` and `GoIndexService`.

1.3 OpenSSL Detection Rule Patterns (Verified Against OpenSSL 3.x Docs)

OpenSSL 3.x consolidated all cryptographic operations under the EVP (Envelope) high-level API. Low-level APIs (`AES_*`, `DES_*`, `SHA_*`) are deprecated but still widely found in legacy code and must also be detected. The following rules cover the primary detection targets.

1.3.1 EVP Symmetric Cipher Rules

Target functions: `EVP_EncryptInit_ex2`, `EVP_EncryptInit_ex`, `EVP_CipherInit_ex2`, `EVP_CIPHER_fetch`

Example detection rule pattern (pseudocode in `DetectionRuleBuilder` style):

```
new DetectionRuleBuilder<CxxTree>()
    .createDetectionRule()
    .forObjectExactTypes("EVP_CIPHER_CTX")
    .forMethods("EVP_EncryptInit_ex2", "EVP_EncryptInit_ex", "EVP_CipherInit_ex2")
    .shouldBeDetectedAs(new AlgorithmFactory<>())
    .withAnyParameters()
    .addDependingDetectionRules(List.of(
        // Capture cipher type arg (e.g. EVP_aes_256_cbc())
        new DetectionRuleBuilder<CxxTree>()
            .createDetectionRule()
            .forMethods("EVP_aes_128_cbc", "EVP_aes_256_cbc", "EVP_aes_128_gcm",
                "EVP_aes_256_gcm", "EVP_aes_128_ctr", "EVP_aes_256_ctr",
                "EVP_chacha20_poly1305", "EVP_des_ede3_cbc")
            .shouldBeDetectedAs(new ValueActionFactory<>("AES")) // or DES, ChaCha20
```

```

))
.buildForContext(new CipherContext<>())

```

EVP_CIPHER_fetch (OpenSSL 3 explicit fetch): In OpenSSL 3, algorithms are fetched by name string : e.g., `EVP_CIPHER_fetch(NULL, "AES-256-GCM", NULL)`. Rules must capture the second string argument as the algorithm identifier.

1.3.2 EVP Digest / Hash Rules

Target: `EVP_DigestInit_ex2`, `EVP_DigestInit_ex`, `EVP_MD_fetch`

```

// Detect: EVP_DigestInit_ex(ctx, EVP_sha256(), NULL)
// Detect: EVP_MD_fetch(NULL, "SHA2-256", NULL)
// Target EVP_MD factory functions: EVP_sha1, EVP_sha224, EVP_sha256,
//   EVP_sha384, EVP_sha512, EVP_sha3_256, EVP_sha3_512,
//   EVP_md5 (deprecated), EVP_md4 (deprecated)

```

1.3.3 EVP Public Key / Asymmetric Rules

Target: `EVP_PKEY_CTX_new`, `EVP_PKEY_CTX_new_id`, `EVP_PKEY_keygen_init`, `EVP_DigestSignInit`

```

// Key type IDs to detect: EVP_PKEY_RSA, EVP_PKEY_EC, EVP_PKEY_DSA,
//   EVP_PKEY_DH, EVP_PKEY_ED25519, EVP_PKEY_ED448, EVP_PKEY_X25519
// For key length: capture RSA_generate_key_ex bits parameter
//   or EVP_PKEY_CTX_set_rsa_keygen_bits()

```

1.3.4 SSL/TLS Context Rules (libssl)

Target: `SSL_CTX_new`, `SSL_CTX_set_cipher_list`, `SSL_CTX_set_ciphersuites`, `TLS_method`, `TLS_client_method`

```

// Detect TLS version via method: TLS_method(), TLSv1_2_method() (deprecated),
//   TLSv1_3_method, DTLS_method, DTLSv1_2_method
// Detect cipher suite strings in SSL_CTX_set_cipher_list()
// SSL_CTX_set_min_proto_version / SSL_CTX_set_max_proto_version

```

1.3.5 Legacy Low-Level API Rules (Deprecated but Common in Legacy Code)

```

// AES: AES_set_encrypt_key, AES_set_decrypt_key, AES_encrypt, AES_cbc_encrypt
// RSA: RSA_public_encrypt, RSA_private_decrypt, RSA_sign, RSA_verify
// SHA: SHA1, SHA256, SHA512 (direct call deprecated in OpenSSL 3)
// HMAC: HMAC() one-shot function
// These are deprecated in OpenSSL 3.0 but must be detected for CBOM accuracy

```

1.3.6 EVP MAC Rules

Target: `EVP_MAC_fetch`, `EVP_MAC_CTX_new`, `EVP_MAC_init`

```

// MAC algorithms: HMAC, CMAC, GMAC, KMAC-128, KMAC-256
// EVP_MAC_fetch(NULL, "HMAC", NULL)

```

```
// EVP_MAC_CTX_set_params() for underlying digest specification
```

Section 2: 12-Week Milestone Timeline

LFX Summer 2026 Dates (Verified): Term runs June 9 – August 29, 2026. Community Bonding: May 28 – June 8 (post-admission, pre-term). Stipend tranches are aligned to milestone completions. Applications open May 14–27, decisions May 28 – June 3.

Community Bonding Period (May 28 – June 8, 2026)

- Read entire codebase: sonar-cryptography (all Maven modules), cbomkit-lib, cbomkit. Clone all repos locally.
- Set up Docker Compose development environment per the repository's instructions. Verify successful scans of Java and Python test projects.
- Engage with maintainers on GitHub Discussions. Introduce yourself in the CBOMkit Discord (discord..org). Review Issue #374 and the chmodshubham fork in detail.
- Contact the sonar-cxx maintainers regarding PR #3013 status and sonar-cxx 2.3.0 release timeline. Assess whether to use the fork SNAPSHOT jar as a temporary dependency.
- Study LANGUAGE_SUPPORT.md and DETECTION_RULE_STRUCTURE.md thoroughly. Map out every interface that the cpp/ module must implement.
- Write a draft architecture doc (1-2 pages) for mentor review before Week 1.

Phase 1: C/C++ Parser Integration (Weeks 1–3: June 9–27)

Week 1 (June 9–13): Maven Scaffolding & Language Registration

- Create cpp/pom.xml with dependency on sonar-cxx (or fork SNAPSHOT). Add cpp module to parent pom.xml.
- Implement CppLanguage.java: key = "c++", name = "C++", file suffixes = [".c", ".cpp", ".cc", ".cxx", ".h", ".hpp"].
- Implement CppCheck.java stub (no rules yet) extending AbstractCheck<CxxTree>.
- Implement CppCheckRegistrar.java stub.
- Wire into CryptographyPlugin.java: register CppCheckRegistrar as an extension.
- Deliverable: mvn compile succeeds for the cpp/ module. Merge to feature branch.

Week 2 (June 16–20): AST Traversal & Symbol Resolution

- Implement CppAstTranslator.java (implements ITranslator<CxxTree>): maps CxxTree AST nodes to the engine's common tree abstraction.
- Validate Symbol class availability from sonar-cxx. If PR #3013 is merged, use the release jar. If not, use the SNAPSHOT.

- Write unit tests for AST traversal using 3 representative .c fixture files with simple EVP_EncryptInit_ex and EVP_DigestInit_ex calls.
- Deliverable: Tests pass; symbol resolution works for simple function calls.

Week 3 (June 23–27): CppSensor, cbomkit-lib Integration & Phase 1 Review

- Implement CppSensor.java extending AbstractSensor. Wires CppCheckRegistrar into the SonarQube sensor lifecycle.
- Implement CppIndexService and CppScannerService in cbomkit-lib (mirrors JavaIndexService and GoIndexService patterns).
- End-to-end test: scan a minimal C project with a single EVP_DigestInit_ex call; verify cbom.json is produced with the correct Algorithm node.
- Milestone Review with mentor. Document any blockers. Write Week 3 progress update to GitHub Discussions.

Phase 2: OpenSSL Detection Module (Weeks 4–7: June 30 – July 25)

Week 4 (June 30 – July 4): EVP Cipher & Digest Rules

- Implement EvpCipherRules.java: cover EVP_EncryptInit_ex, EVP_EncryptInit_ex2, EVP_CipherInit_ex2, EVP_CIPHER_fetch plus all EVP_aes_*_cbc/gcm/ctr, EVP_chacha20_poly1305.
- Implement EvpDigestRules.java: cover EVP_DigestInit_ex, EVP_DigestInit_ex2, EVP_MD_fetch plus EVP_sha* and legacy EVP_md5, EVP_sha1.
- Write JUnit 5 tests using fixture .c files: minimum 2 test files per rule class.
- Deliverable: EvpCipherRules and EvpDigestRules tested and passing.

Week 5 (July 7–11): EVP Public Key & Key Generation Rules

- Implement EvpPkeyRules.java: detect EVP_PKEY_CTX_new_id with EVP_PKEY_RSA/EC/ED25519 IDs, EVP_PKEY_keygen_init, EVP_DigestSignInit for RSA/ECDSA signing.
- Implement EvpKeyGenRules.java: capture key length parameters from RSA_generate_key_ex (bits argument), EVP_PKEY_CTX_set_rsa_keygen_bits, EC curve names via EC_KEY_new_by_curve_name.
- Write tests covering RSA-2048, RSA-4096, EC P-256, EC P-384, ED25519 key generation patterns.

Week 6 (July 14–18): SSL/TLS Context Rules & MAC Rules

- Implement SslCtxRules.java: detect SSL_CTX_new with TLS_method/TLS_client_method/TLS_server_method, SSL_CTX_set_cipher_list (capture cipher string), SSL_CTX_set_ciphersuites, SSL_CTX_set_min/max_proto_version.

- Implement `EvpMacRules.java`: detect `EVP_MAC_fetch` for HMAC/CMAC/GMAC/KMAC, `EVP_MAC_CTX_new`, `EVP_MAC_init`.
- Implement `LegacyRules.java`: detect deprecated `AES_set_encrypt_key`, `RSA_public_encrypt`, `SHA256 (direct)`, `HMAC()` one-shot.

Week 7 (July 21–25): Coverage Audit, Edge Cases & Phase 2 Review

- Run coverage audit against the full OpenSSL 3.x EVP API surface. Target: 80%+ of commonly-used cryptographic functions detected.
- Handle edge cases: `EVP_CIPHER_fetch` / `EVP_MD_fetch` with string algorithm names (OpenSSL 3 explicit fetch pattern). These require string literal extraction from the AST.
- Verify mapper/ correctly translates C-detected nodes into Algorithm/Protocol/Key INode hierarchy for CBOM output.
- Milestone Review with mentor. Write mid-program blog post draft.

Phase 3: Integration, Validation & Documentation (Weeks 8–12: July 28 – Aug 29)

Week 8 (July 28 – Aug 1): Real-World Project Validation

- Scan 3–5 real-world C/C++ projects using OpenSSL (see Section 3 for selected projects). Use `cbomkit-lib` or Docker Compose setup.
- Compare detected CBOM against manually-inventoried cryptographic calls. Calculate precision and recall.
- File GitHub issues for any missed detections (false negatives) or incorrect detections (false positives).

Week 9 (Aug 4–8): OPA Policy Test Suite

- Write 3+ Rego policy files for CBOMkit's OPA integration (see Section 5 for full examples).
- Test policies against CBOMs generated from the validation projects.
- Verify CBOMkit's `/api/v1/compliance/check` endpoint correctly evaluates C/C++ CBOMs.

Week 10 (Aug 11–15): PR Submission & Code Review

- Open PR against `cbomkit/sonar-cryptography` with the full `cpp/` module. Address code review feedback from maintainers.
- Open separate PR against `cbomkit/cbomkit-lib` with `CppIndexService` and `CppScannerService`.
- Ensure all files pass Spotless Maven plugin (code formatting) checks as used in the existing codebase.

Week 11 (Aug 18–22): Research Document & Library Survey

- Write the library research document (see Section 4 table). Cover wolfSSL, libsodium, BoringSSL, Botan, and Rust ring/rustls with parsing feasibility analysis.
- Document the sonar-cxx Symbol dependency resolution process for future maintainers.
- Finalize test suite documentation in cpp/README.md.

Week 12 (Aug 25–29): Blog Post, Documentation & Final Submission

- Publish technical blog post on .org (or GitHub Discussions) covering the C/C++ integration approach, key technical challenges, and OpenSSL detection coverage achieved.
- Update LANGUAGE_SUPPORT.md with C/C++ entry. Update main repository README.
- Submit final mentorship report to LFX platform. All deliverables tagged in GitHub.

Timeline Summary Table

Phase	Weeks	Dates	Key Deliverables	Milestone
Community Bonding	Pre	May 28–Jun 8	Codebase study, env setup, community intro	Architecture draft
Phase 1: Parser Integration	1–3	Jun 9–27	cpp/ module, AST translator, CppSensor, cbomkit-lib services	End-to-end scan works
Phase 2: OpenSSL Detection	4–7	Jun 30–Jul 25	All 6 rule files, 80%+ EVP API coverage, test fixtures	Coverage audit passed
Phase 3: Integration & Docs	8–12	Jul 28–Aug 29	Validation scans, OPA policies, PRs merged, blog post	Final submission

Section 3: Validation Projects

The following five real-world C/C++ projects are recommended for validating the detection engine. Each was selected for a specific quality: diversity of OpenSSL API usage, production deployment scale, and coverage of different crypto pattern families.

1. curl (<https://github.com/curl/curl>)

- Why valuable: curl is one of the most widely deployed C projects globally. It uses OpenSSL for TLS (SSL_CTX_new, SSL_CTX_set_cipher_list), certificate verification, and digest operations. Code is well-maintained and modular.
- Expected detection targets: SSL_CTX_new, TLS_client_method, SSL_CTX_set_cipher_list, EVP_DigestInit_ex (for SHA-256 digest verification), X509_verify_cert patterns.
- Success metric: >85% of cryptographic API calls in lib/vtls/openssl.c detected and mapped to CBOM nodes.

2. OpenSSH (<https://github.com/openssh/openssh-portable>)

- Why valuable: OpenSSH uses OpenSSL for key generation (RSA, ECDSA, ED25519), symmetric ciphers (AES-CTR, AES-GCM, ChaCha20-Poly1305), and MACs (HMAC-SHA2). It represents a high-value crypto-intensive production codebase.
- Expected detection targets: EVP_PKEY_keygen_init with RSA/EC key types, EVP_DigestSignInit for signing, EVP_EncryptInit_ex for session ciphers, EVP_MAC_init for HMAC-SHA2.
- Success metric: Key exchange and cipher negotiation patterns detectable; key types and lengths captured in CBOM.

3. nginx (<https://github.com/nginx/nginx>)

- Why valuable: nginx is a production-grade web server using OpenSSL for TLS termination. It exercises SSL_CTX configuration, certificate loading, and session ticket handling.
- Expected detection targets: SSL_CTX_new, SSL_CTX_use_certificate_file, SSL_CTX_set_cipher_list, SSL_CTX_set_min_proto_version.
- Success metric: TLS configuration assets (protocol version, cipher suites) appear in CBOM. Useful for OPA policy testing.

4. PostgreSQL (<https://github.com/postgres/postgres>)

- Why valuable: PostgreSQL uses OpenSSL for server-side TLS, password hashing (MD5, SHA-256 for SCRAM-SHA-256), and SSL certificate management. Mix of legacy and modern API usage.
- Expected detection targets: SSL_CTX_new in backend/libpq/be-secure-openssl.c, EVP_DigestInit_ex for SCRAM authentication, legacy MD5 calls.

- Success metric: Detects both modern EVP and legacy digest usage; demonstrates mixed-API detection capability.

5. libgit2 (<https://github.com/libgit2/libgit2>)

- Why valuable: libgit2 is a portable C implementation of git that uses OpenSSL for HTTPS transport (via libcurl or directly), SHA-1/SHA-256 for object hashing, and HMAC for authentication tokens.
- Expected detection targets: EVP_DigestInit_ex for SHA-1/SHA-256 object IDs, HMAC patterns for smart HTTP authentication.
- Success metric: Hash algorithm and MAC patterns detected; demonstrates utility beyond TLS-centric codebases.

Project	Primary Crypto Use	OpenSSL API Families	Detection Priority
curl	HTTPS/TLS transport	SSL_CTX_*, EVP_Digest*	High : TLS protocol detection
OpenSSH	Key gen, session ciphers, MACs	EVP_PKEY_*, EVP_Encrypt*, EVP_MAC*	High : PQ readiness check
nginx	TLS termination	SSL_CTX_*, TLS_method	High : OPA policy testing
PostgreSQL	TLS + password hashing	SSL_CTX_*, EVP_Digest*, legacy MD5	Medium : legacy API coverage
libgit2	Object hashing, HMAC	EVP_Digest*, HMAC()	Medium : non-TLS crypto patterns

Section 4: Library Research Table : Future Integration Feasibility

This table evaluates five additional cryptographic libraries as candidates for future sonar-cryptography detection modules. Scores are assessed based on: library market share, availability of a SonarQube-compatible AST parser, breadth of cryptographic API coverage, and post-quantum algorithm support.

Library	Language	Popularity (1–10)	Parsing Difficulty (1–10)	Coverage Potential (1–10)	PQ Crypto Support	Key Notes
wolfSSL	C	7	6	8	YES : ML-KEM (FIPS 203), ML-DSA (FIPS 204), Falcon, SLH-DSA (FIPS 205), LMS, XMSS. Note: NTRU was deprecated and removed by wolfSSL after NIST did not standardise it.	Widely used in embedded/IoT systems. API mirrors OpenSSL in 'compatibility mode'. Can reuse C sonar-cxx parser from C/C++ module. wolfCrypt provides a distinct lower-level API.
libsodium	C	8	5	7	None : all current algorithms (X25519, Ed25519, XSalsa20-Poly1305) are classically vulnerable to quantum computers via Shor's algorithm. No NIST PQC support.	High-level, opinionated API (crypto_box, crypto_sign, crypto_secretbox). Fixed algorithm choices simplify detection : fewer parameters to track. Language bindings exist for Python, Node; future multi-language coverage possible.
BoringSS	C/C++	7	7	7	Yes :	Google fork of OpenSSL. API

L					X25519Kyber768 hybrid (Chrome/Android TLS). FIDO2.	is largely compatible but diverges in SSL_CTX and internal types. Hybrid PQ TLS support is production-grade. Requires its own rule set despite OpenSSL similarity. No stable release versioning : tracks Chromium.
Botan	C++	5	8	9	Yes : ML-KEM (Kyber), ML-DSA (Dilithium), SPHINCS+, Classic McEliece	Richest PQ support of any C++ library. C++ class hierarchy (e.g., Cipher_Mode, HashFunction, MessageAuthenticationCode) maps naturally to INode types. SonarQube C++ parsing via sonar-cxx applies. Most relevant for post-quantum CBOM research.
Rust ring / rustls	Rust	8	9	7	Partial : rustls supports X25519; ring does not include NIST PQC as of 2026	SonarQube released first-party Rust support in April 2025 (no longer experimental). However, it uses Clippy as a backend rather than the CheckRegistrar pattern used by Java/Python/Go modules, making sonar-cryptography integration architecturally distinct. ring is a minimal, audited crypto library; rustls is a pure-Rust TLS stack used by curl and Tokio. Parsing difficulty remains highest due to Rust's ownership and type system. Priority: Low for 2027, High for 2028+.

Recommendation for Mentorship Scope: Focus on OpenSSL 3.x (primary deliverable) and document wolfSSL as the highest-value secondary target given its API compatibility mode. Botan is the highest-value target for post-quantum research follow-up.

Section 5: OPA Policy Testing : Three Verified Examples

CBOMkit integrates Open Policy Agent (OPA) for compliance evaluation via the `/api/v1/compliance/check` endpoint and the `CBOMKIT_REGULATOR_API_BASE` configuration. Policies are written in Rego, OPA's declarative language. The following three policies test real security requirements against C/C++ CBOMs.

Policy 1: Cryptographic Agility Check

This policy flags any code that hardcodes a cipher algorithm string instead of parameterizing it : a common anti-pattern in C/C++ that prevents easy algorithm migration (critical for post-quantum transition planning).

```
package cbomkit.cryptography

# Crypto agility check: flag direct algorithm string literals in EVP_CIPHER_fetch /
# EVP_MD_fetch
# rather than constants or configuration-driven selection.

import future.keywords.contains
import future.keywords.if

deny contains msg if {
    component := input.components[_]
    component.type == "cryptographic-asset"
    component.cryptoProperties.assetType == "algorithm"
    # NOTE: The additionalContext field in real CBOM output stores the API call
    # signature
    # (e.g. "javax.crypto.Cipher#getInstance"), not a parameter quality flag.
    # This line is a PROPOSED SCHEMA EXTENSION : not yet in CycloneDX v1.6.
    # A future enrichment step could tag string-literal algorithm names.
    component.evidence.occurrences[_].additionalContext == "hardcoded-literal"
    msg := sprintf(
        "Hardcoded algorithm literal detected at %v:%v : use configurable algorithm
        selection for crypto agility.",
        [component.evidence.occurrences[0].location,
        component.evidence.occurrences[0].line]
    )
}
```

Note on Policy 1: The `additionalContext == "hardcoded-literal"` check is a proposed extension to the CycloneDX v1.6 CBOM schema. In current CBOM output, `additionalContext` stores the API call signature (e.g., `"javax.crypto.Cipher#getInstance"`). A future enrichment step during C/C++

scanning could tag statically-detected string literals with this marker, enabling the policy to fire. This policy is included as a design target for the cpp/ module's enricher integration : not a feature of the current CycloneDX schema. Policies 2 and 3 below are immediately testable against real CBOM output.

Policy 2: Deprecated Algorithm Ban

This policy enforces that no deprecated or weak cryptographic algorithms appear in the CBOM. It covers algorithms deprecated in OpenSSL 3.0 (MD5, SHA-1 for signatures, DES, 3DES, RC4) and NIST-deprecated algorithms.

```
package cbomkit.cryptography

# Deprecated algorithm ban policy
# Covers: MD5, SHA-1 (signatures), DES, 3DES (Triple-DES), RC4, RC2, Blowfish

banned_algorithms := {
    "MD5", "MD4", "MD2",
    "SHA-1", "SHA1",          # Allowed in HMAC but not signatures
    "DES", "3DES", "DESEde",
    "RC4", "RC2",
    "Blowfish",
}

deny contains msg if {
    component := input.components[_]
    component.type == "cryptographic-asset"
    component.cryptoProperties.assetType == "algorithm"
    alg_name := upper(component.cryptoProperties.algorithmProperties.name)
    banned_algorithms[alg_name]
    msg := sprintf(
        "Deprecated algorithm '%v' detected at %v : replace with an approved
algorithm.",
        [component.cryptoProperties.algorithmProperties.name,
         component.evidence.occurrences[0].location]
    )
}
```

Test scenario: Scan nginx compiled with a legacy cipher suite string containing RC4-SHA. The CBOM will include an Algorithm component for RC4. The policy fires, flagging the file and line of SSL_CTX_set_cipher_list.

Policy 3: Key Length Requirement

This policy enforces minimum key lengths by algorithm family. It implements NIST SP 800-131A Rev. 2 minimums: RSA $\geq$ 2048 bits, ECC $\geq$ 224 bits, AES $\geq$ 128 bits.

```
package cbomkit.cryptography

# Key length requirements per NIST SP 800-131A Rev. 2

minimum_key_lengths := {
  "RSA": 2048,
  "EC": 224,
  "DSA": 2048,
  "DH": 2048,
  "AES": 128,
}

deny contains msg if {
  component := input.components[_]
  component.type == "cryptographic-asset"
  component.cryptoProperties.assetType == "algorithm"
  props := component.cryptoProperties.algorithmProperties
  min_len := minimum_key_lengths[upper(props.name)]
  props.keyLength < min_len
  msg := sprintf(
    "Key length %v bits for %v is below NIST minimum of %v bits. Location: %v",
    [props.keyLength, props.name, min_len,
     component.evidence.occurrences[0].location]
  )
}
```

Test scenario: A C program calling `RSA_generate_key_ex` with `bits=1024`. The CBOM captures `keyLength=1024` for the RSA algorithm component. The policy fires with a clear remediation message. A 2048-bit or 4096-bit key passes silently.

Section 6A: Cover Letter

To the CBOMkit Mentorship Selection Committee,

I am writing to apply for the LFX / Summer 2026 mentorship: 'Extending Language and Library Support in CBOMkit.' This project sits at the precise intersection of the three areas I care most about: applied cryptography, static analysis tooling, and open-source infrastructure for post-quantum migration.

My background combines hands-on experience with C/C++ systems programming, Java-based tooling development, and cryptographic protocols. I have contributed to open-source projects involving AST analysis and have studied the CycloneDX CBOM specification and the SonarQube plugin API in preparation for this application. I have read the `LANGUAGE_SUPPORT.md` and `DETECTION_RULE_STRUCTURE.md` documentation in full, studied the existing `java/`, `python/`, and `go/` modules in the repository, and carefully reviewed Issue #374 where community contributor `chmodshubham` documented a proof-of-concept C/C++ implementation.

My unique angle on this project is the recognition that the critical dependency : Symbol class support in `sonar-cxx` : is already in progress via PR #3013, and that a prior implementation reaching approximately 90% OpenSSL coverage exists in a fork. I understand that my primary job is not to start from zero, but to engage with the `sonar-cxx` maintainers, build production-quality detection rules with test coverage, integrate with `cbomkit-lib`, validate against real projects, and deliver the PRs that get merged. This requires engineering discipline and community collaboration, not just code.

The post-quantum cryptography migration problem is urgent and real. NIST finalized ML-KEM, ML-DSA, and SLH-DSA in 2024. The majority of C/C++ codebases : including `curl`, `OpenSSH`, `nginx`, `PostgreSQL`, and thousands of embedded systems : use OpenSSL as their cryptographic foundation. Without tooling to discover and inventory these cryptographic assets, organizations cannot plan their migration to quantum-safe algorithms. CBOMkit is precisely this tooling, and extending it to C/C++ is the single highest-impact contribution I could make in a summer mentorship.

I commit to weekly progress updates on GitHub Discussions, full test coverage for every detection rule I write, prompt responsiveness to code review feedback, and a technical blog post published on the website documenting everything I learn. I look forward to contributing to this community.

Respectfully,

[Applicant Name]

Section 6B: Technical Proposal

Overview & Objective

This proposal describes a plan to add production-quality C/C++ language support and OpenSSL 3.x detection rules to the cbomkit/sonar-cryptography plugin, enabling the CBOMkit toolchain to generate Cryptographic Bills of Materials (CBOMs) from C and C++ source code. C/C++ represents a dominant share of production cryptographic code globally, and its absence from CBOMkit is the most significant gap in the current tool's coverage.

Technical Approach

The implementation follows the documented pattern in LANGUAGE_SUPPORT.md exactly. The sonar-cryptography architecture is modular by design: adding C/C++ requires (a) a new cpp/ Maven submodule implementing the four required generics via sonar-cxx, (b) OpenSSL-specific detection rules using the DetectionRuleBuilder DSL, (c) integration of CppIndexService and CppScannerService into cbomkit-lib, and (d) comprehensive test fixtures.

The critical dependency is sonar-cxx's Symbol class, which provides type resolution for C/C++ ASTs. Community contributor chmodshubham raised PR #3013 to SonarOpenCommunity/sonar-cxx in February 2026. My first action will be to engage with those maintainers and, if necessary, develop against the fork's SNAPSHOT jar while the PR is reviewed, ensuring the mentorship timeline is not blocked.

OpenSSL 3.x unified all cryptographic operations under the EVP (Envelope) high-level API. Detection rules will cover: symmetric ciphers (EVP_EncryptInit_ex, EVP_CIPHER_fetch with algorithm name strings), message digests (EVP_DigestInit_ex, EVP_MD_fetch), public key operations (EVP_PKEY_CTX_new_id with key type constants), SSL/TLS context configuration (SSL_CTX_new, SSL_CTX_set_cipher_list, TLS_method), MACs (EVP_MAC_fetch, EVP_MAC_init), and legacy deprecated APIs (AES_*, RSA_*, SHA256 direct calls) which remain prevalent in older codebases.

Risk Analysis & Mitigations

Risk	Likelihood	Impact	Mitigation
sonar-cxx PR #3013 not merged before mentorship start	Medium	High	Use the fork's SNAPSHOT jar as a local Maven dependency. This unblocks implementation while the upstream PR is reviewed.
sonar-cxx AST node types differ from sonar-java/sonar-go	Medium	Medium	Study ITranslator implementations in existing modules. Write comprehensive unit tests for AST traversal before writing detection rules.

abstractions			
EVP_CIPHER_fetch string-based algorithm names are complex to detect	High	Medium	Handle string literals via Value extraction in DetectionRuleBuilder. Document cases where dynamic strings cannot be statically resolved.
Macro-heavy OpenSSL code confuses static analysis	Medium	Low	sonar-cxx uses a preprocessor expansion step. Test with #define-heavy headers. Document limitations in the module README.
cbomkit-lib SensorContextTester approach breaks for C++	Low	High	Study the existing cbomkit-lib test infrastructure. The SensorContextTester pattern is language-agnostic; the only language-specific part is the scanner dependency.

Success Metrics

See Section 7 for detailed measurable outcomes. At a high level:

- All detection rule classes pass JUnit 5 tests with coverage of the primary OpenSSL 3.x EVP API functions.
- 80%+ of cryptographic API calls in curl's lib/vtls/openssl.c are correctly detected and represented in the CBOM.
- PRs submitted to cbomkit/sonar-cryptography and cbomkit/cbomkit-lib before the end of Week 10.
- Three OPA policy tests pass against CBOMs generated from validation projects.
- Blog post published on website by the final week.

Why This Mentorship Is the Right Fit for Me

I have the technical skills for this project (C/C++ systems programming, Java tooling, cryptographic APIs) and the open-source experience to collaborate effectively with the CBOMkit maintainers. The prior work in Issue #374 gives me a concrete foundation rather than a blank slate. The 12-week timeline is achievable because the architecture is well-documented, the patterns are established by the Java/Python/Go modules, and the primary blocker (sonar-cxx Symbol support) has a clear path to resolution. I am committed to delivering merged, production-quality code : not just a prototype.

Section 6C: Good First Contribution

Immediate Action: Engage on Issue #374 and Audit sonar-cxx 2.3.0 Readiness

The most valuable immediate contribution I can make before the mentorship formally begins is to post a detailed technical comment on Issue #374 that:

1. Confirms I have reproduced the chmodshubham fork locally and verified that the sonar-cxx SNAPSHOT jar (with PR #3013 changes) allows the cpp/ module to compile and produce an AST.
2. Documents which sonar-cxx methods and interfaces are required by sonar-cryptography's engine/ module (R, T, P, S generics) with exact Java interface names, so sonar-cxx maintainers have a precise requirements checklist for their 2.3.0 milestone.
3. Proposes a minimal test case (.c file + expected CBOM output) that the community can use to validate any sonar-cxx release for compatibility with sonar-cryptography.

This is a concrete, non-trivial contribution that requires actually reading both codebases, running the implementation, and synthesizing findings. It establishes credibility with maintainers, unblocks the merge path for sonar-cxx 2.3.0, and is directly useful to the project regardless of whether I am selected as the mentee.

Potential PR: Add C/C++ File Extension to Language Support Documentation

A lower-complexity but immediately mergeable PR would update LANGUAGE_SUPPORT.md to add a C/C++ placeholder section describing:

- The four required generic classes and their sonar-cxx equivalents once PR #3013 is merged.
- The blocking status and sonar-cxx 2.3.0 dependency.
- A 'coming soon' note with a link to Issue #374.

This PR touches documentation only, carries no risk of breaking existing functionality, and gives maintainers a chance to evaluate my writing quality and understanding of the codebase before the coding work begins.

Section 7: Success Metrics

The following table defines measurable success criteria for each deliverable. Metrics are designed to be objective and verifiable by mentors at each milestone review.

Deliverable	Success Metric	Verification Method	Target Date
sonar-cxx Symbol dependency resolved	sonar-cxx 2.3.0 jar (or fork SNAPSHOT) used as Maven dependency; CppCheck.java compiles with no errors	mvn compile -pl cpp succeeds in CI	Week 2 (Jun 20)
cpp/ module scaffolding	CryptographyPlugin registers CppCheckRegistrar; SonarQube Docker instance recognizes .c/.cpp files as C/C++ language	sonarqube/projects page shows C/C++ language for a test project	Week 3 (Jun 27)
End-to-end basic scan	Scanning a minimal .c file with one EVP_DigestInit_ex call produces a valid cbom.json with one Algorithm component	jq .components[0].type cbom.json == "cryptographic-asset"	Week 3 (Jun 27)
EvpCipherRules test coverage	JUnit tests cover: EVP_aes_128_cbc, EVP_aes_256_cbc, EVP_aes_128_gcm, EVP_aes_256_gcm, EVP_aes_256_ctr, EVP_chacha20_poly1305, EVP_CIPHER_fetch with string literal	mvn test -pl cpp passes; 7+ test methods in EvpCipherRulesTest.java	Week 4 (Jul 4)
EvpDigestRules test coverage	Tests cover: EVP_sha256, EVP_sha512, EVP_sha3_256, EVP_md5, EVP_MD_fetch with string	5+ test methods in EvpDigestRulesTest.java	Week 4 (Jul 4)
EvpPkeyRules test coverage	Tests cover: RSA-2048, RSA-4096, EC P-256, EC P-384, ED25519 key generation; key length captured in CBOM	KeyLength field populated in CBOM component; 5+ test methods	Week 5 (Jul 11)
SslCtxRules test coverage	Tests cover: TLS_method, TLS_client_method, SSL_CTX_set_cipher_list (cipher string captured),	Protocol component appears in CBOM; 4+ test methods	Week 6 (Jul 18)

	protocol version detection		
LegacyRules test coverage	Tests cover: AES_set_encrypt_key, RSA_public_encrypt, SHA256 direct call, HMAC()	Deprecated API calls produce CBOM components with deprecated=true annotation	Week 6 (Jul 18)
OpenSSL 3.x EVP API coverage audit	At least 80% of the primary EVP API functions in evp.h produce detectable CBOM output	Coverage audit document listing detected vs. undetected functions	Week 7 (Jul 25)
curl validation	85%+ of cryptographic API calls in lib/vtls/openssl.c detected (manually verified sample of 20 calls)	Manual comparison of CBOM vs. grep-based inventory of openssl.c	Week 8 (Aug 1)
OpenSSH validation	Key type (RSA/EC/ED25519) and key length correctly captured for 5+ key generation call sites	CBOM keyLength fields match source code parameters	Week 8 (Aug 1)
OPA Policy 1 (Crypto Agility)	Policy correctly fires for hardcoded literal algorithm names; does not fire for config-driven selection	opa eval with test CBOM files returns expected deny/allow	Week 9 (Aug 8)
OPA Policy 2 (Deprecated Ban)	Policy fires for MD5, SHA-1, DES, RC4; does not fire for AES-256- GCM, SHA-256	opa eval test suite: 6 positive cases, 2 negative cases	Week 9 (Aug 8)
OPA Policy 3 (Key Length)	Policy fires for RSA- 1024, EC-160; does not fire for RSA-2048, EC- 256	opa eval test suite: 4 positive cases, 2 negative cases	Week 9 (Aug 8)
PR submitted to sonar-cryptography	PR opened against cbomkit/sonar- cryptography main branch; CI passes; at least one maintainer review requested	GitHub PR URL visible; Actions checks green	Week 10 (Aug 15)
PR submitted to cbomkit-lib	PR opened against cbomkit/cbomkit-lib with CppIndexService and CppScannerService	GitHub PR URL visible; integration tests pass	Week 10 (Aug 15)
Library research document	Document covers wolfSSL, libsodium, BoringSSL, Botan, ring/rustls with feasibility scoring	Document committed to repository as docs/LIBRARY_RESEAR CH.md	Week 11 (Aug 22)
LANGUAGE_SUPP ORT.md updated	C/C++ entry added with sonar-cxx version requirement, file suffixes, and link to detection rules	PR merged or approved	Week 11 (Aug 22)
Blog post published	Technical blog post of at least 800 words published on website or	Public URL accessible	Week 12 (Aug 29)

	GitHub Discussions covering architecture, challenges, and results		
--	---	--	--



End of Application Document | cbomkit/sonar-cryptography | LFX / Summer 2026